

ReflectionPad1d#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.padding.ReflectionPad1d.html>

Description:

Pads the input tensor using the reflection of the input boundary. For N-dimensional padding, use `torch.nn.functional.pad()`. `padding` (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 2-tuple, uses `(padding_left\text{padding\_left}padding_left, padding_right\text{padding\_right}padding_right)` Note that padding size should be less than the corresponding input dimension. Input: (C, W_{in}) or (N, C, W_{in}) . Output: (C, W_{out}) or (N, C, W_{out}) , where

Function Signature

```
class torch.nn.modules.padding.ReflectionPad1d(padding)
[source]#
```

Parameters

Shape:

Code Examples

```
>>> m = nn.ReflectionPad1d(2)
>>> input = torch.arange(8, dtype=torch.float).reshape(1, 2, 4)
>>> input
tensor([[[0., 1., 2., 3.],
         [4., 5., 6., 7.]]])
>>> m(input)
tensor([[[2., 1., 0., 1., 2., 3., 2., 1.],
         [6., 5., 4., 5., 6., 7., 6., 5.]]])
>>> # using different paddings for different sides
>>> m = nn.ReflectionPad1d((3, 1))
>>> m(input)
tensor([[[3., 2., 1., 0., 1., 2., 3., 2.],
         [7., 6., 5., 4., 5., 6., 7., 6.]]])
```

Detailed Documentation

Documentation

Pads the input tensor using the reflection of the input boundary.

For N-dimensional padding, use `torch.nn.functional.pad()`.

padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 2-tuple, uses $(padding_left, padding_right)$. Note that padding size should be less than the corresponding input dimension.

Input: (C, W_{in}) or (N, C, W_{in})

Output: (C, W_{out}) or (N, C, W_{out}) , where

$$W_{out} = W_{in} + padding_left + padding_right$$